Lab 11 — Cross-Site Scripting (XSS)

> **SY0-701 Objective 2.3** — Explain various types of vulnerabilities
>
> **Domain 2** — Threats, Vulnerabilities, and Mitigations (22%) · **Time:** ~15 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: make a page reflect attacker-controlled markup, then stop it with output encoding and a Content-Security-Policy header.

You'll build: a vulnerable reflected-XSS page and its encoding + CSP mitigations.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1.

> **Legal note:** Everything runs on `127.0.0.1` in your own playground. Testing XSS against sites you do not own breaches Singapore's Computer Misuse Act.

Background

- **XSS** injects script that runs in another user's browser *inside the trusted site's own origin*, so the browser treats it as first-party code.
- **Reflected** XSS rides in the request and is echoed straight back; **stored** XSS is saved and fires for every later visitor; **DOM-based** XSS never reaches the server.
- Typical impact: session-cookie theft, forced actions as the victim, and phishing overlays.
- Defences layer up: context-aware **output encoding** (`htmlspecialchars`), cookie hardening (`HttpOnly`, `Secure`, `SameSite`), and a **Content-Security-Policy** header.

Steps

1. Install PHP

```
apt-get update -qq && apt-get install -y php-cli curl
```

2. Create a vulnerable reflected page

The page echoes the `q` parameter straight into the HTML body with no encoding.

```
cat > search.php <<'EOF'
<?php
$q = $_GET['q'] ?? '';
echo "<h1>Results for: $q</h1>";   /* VULNERABLE: raw echo */
EOF
```

3. Start the local server

```
php -S 127.0.0.1:8080 -t . >/tmp/php.log 2>&1 &
sleep 1 && echo "server up"
```

4. Trigger the reflected payload

The injected markup is returned verbatim, so the browser would render (and execute) it.

```
curl -s 'http://127.0.0.1:8080/search.php?q=<b>INJECTED</b>'
```

```
<h1>Results for: <b>INJECTED</b></h1>      # markup reflected, not escaped
```

5. Mitigate with output encoding

`htmlspecialchars` converts `<` `>` `"` `&` into entities so the browser shows text, not tags.

```
cat > search.php <<'EOF'
<?php
$q = $_GET['q'] ?? '';
echo "<h1>Results for: " . htmlspecialchars($q, ENT_QUOTES, 'UTF-8') . "</h1>";
EOF
```

6. Add a Content-Security-Policy header

A router script sets a strict CSP as a defence-in-depth layer that blocks injected remote script.

```
cat > router.php <<'EOF'
<?php
header("Content-Security-Policy: default-src 'self'");
header("X-Content-Type-Options: nosniff");
require __DIR__ . '/search.php';
EOF
pkill -f 'php -S' 2>/dev/null; sleep 1
php -S 127.0.0.1:8080 router.php >/tmp/php.log 2>&1 &
sleep 1 && echo "server restarted with CSP"
```

7. Confirm encoding and the CSP header

```
curl -sD - 'http://127.0.0.1:8080/?q=<b>INJECTED</b>' | grep -i 'content-security-policy'
curl -s  'http://127.0.0.1:8080/?q=<b>INJECTED</b>'
```

```
Content-Security-Policy: default-src 'self'
<h1>Results for: &lt;b&gt;INJECTED&lt;/b&gt;</h1>   # now escaped text
```

Test it

Confirm the payload is neutralised and the header is served.

```
OUT=$(curl -sD /tmp/h.txt 'http://127.0.0.1:8080/?q=<b>x</b>')
echo "$OUT" | grep -q '&lt;b&gt;' && echo "ENCODING OK — markup shown as text"
grep -qi 'content-security-policy' /tmp/h.txt && echo "CSP HEADER OK"
```

Expected: the reflected markup appears escaped (`<b>`) and the response carries a `Content-Security-Policy` header.

Troubleshooting

Symptom	Fix
Still see raw `<b>` after step 5	The old server is still running; `pkill -f 'php -S'` and restart.
No CSP header	Requests must go through `router.php` (hit `/` not `/search.php`).
`curl` strips the `<` `>`	Quote the URL in single quotes as shown, or URL-encode as `%3C` `%3E`.

Challenge

Add a stored-XSS page (a guestbook that saves messages to a file) and show that output encoding at render time fixes both reflected and stored variants. Then set `HttpOnly` on a session cookie and explain which XSS impact that blocks.

Reflection

Objective 2.3 lists XSS as a web-based vulnerability. Why does XSS defeat the browser's same-origin policy, and why is encoding on *output* more reliable than filtering on *input*?

Exam tips

- Reflected rides in the request; stored is persisted and hits every visitor; DOM-based never reaches the server.
- Encode on output for the context you land in (HTML body, attribute, JS, URL differ); blocking the word "script" is trivially bypassed.
- `HttpOnly` blocks `document.cookie` theft; `SameSite` mitigates CSRF chained with XSS.
- CSP is defence in depth; avoid `unsafe-inline`, which undermines the whole policy.

References

- OWASP Cross-Site Scripting — https://owasp.org/www-community/attacks/xss/
- OWASP XSS Prevention Cheat Sheet — https://cheatsheetseries.owasp.org
- CWE-79 Cross-site Scripting — https://cwe.mitre.org/data/definitions/79.html
- MDN Content-Security-Policy — https://developer.mozilla.org/docs/Web/HTTP/CSP
